47

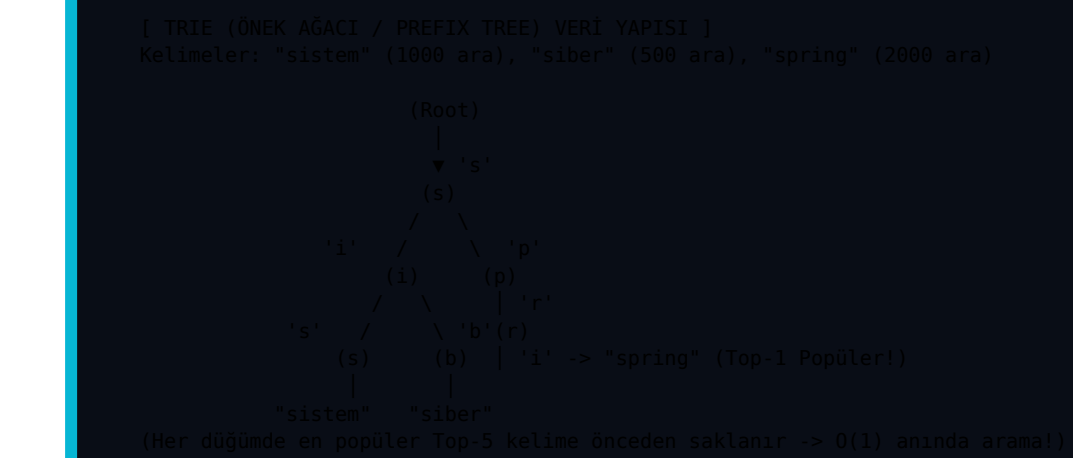
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Arama Otomatik Tamamlama (Typeahead / Google Suggestion): Trie ve Redis ZSET

Designing Search Typeahead (Google Suggestion): Trie & Redis ZSET

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Google'ın arama kutusuna 'sis' yazdığınız anda kutunun altında 'sistem tasarımı', 'siber güvenlik' gibi önerilerin şak diye belirmesini düşünün. Dünyada 1 milyar kelime varken Google her harf bastığınızda tüm veritabanını tarayamaz! Bunun yerine akıllı bir soy ağacı (Trie - Önek Ağacı) kurar. 's' harfine basınca 's' ile başlayan tüm dalları, 'i' harfine basınca 'si' ile başlayan dalları görür. Üstelik her dalın tepesinde en çok aranan ilk 5 kelime altın tepside hazır tutulur. Siz tuşa bastığınız anda 2 milisaniyede liste önünüze dökülür!

🇬🇧 ENGLISH (ELI5)

The instant you type 'sys' into Google, a dropdown flashes: 'system design', 'system requirements'. With a corpus of billions of search terms, Google cannot run a database SQL scan on every keystroke! Instead, it navigates an in-memory Prefix Tree (Trie). Typing 's' lands on the 's' branch; typing 'y' traverses to 'sy'. Crucially, every tree node pre-caches the Top-5 most popular global search phrases right on its branch. The moment your finger hits the key, suggestions render in 2 milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Typeahead Mimarisi: 1) Trie Veri Yapısı: Kökten yapraklara karakter bazlı dallanan ağaçtır. Arama süresi kelime uzunluğu k kadardır ($O(k)$); kelime sayısından (N) tamamen bağımsızdır. 2) Top-K Önbellekleme: Her Trie düğümünde en sık aranan ilk 5-10 kelime ve frekansı saklanır. Kullanıcı 'sis' yazdığında ağacın derinliklerine inmeden o düğümdeki hazır listeyi döner ($O(1)$). 3) Çevrimdışı Analiz Hattı (Offline Aggregation Pipeline): Trie çalışma anında güncellenmez (yavaşlatmamak için). Arama logları Kafka -> Flink/Spark ile toplanır; gece batch işinde yeni frekanslar hesaplanır ve güncel Trie bellek kümesine (Redis veya Custom C++ / Java servisi) yüklenir. 4) İstemci Tarafı Optimizasyon: Tarayıcı tarafında 200ms `debounce` (kullanıcı yazmayı bırakana kadar bekleme) ve yerel önbellek uygulanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Search Typeahead Architecture: 1) The Trie Data Structure: Character-level prefix tree providing $O(L)$ traversal time (L = query length), completely independent of total vocabulary size N. 2) Node-Level Top-K Caching: To eliminate subtree traversals during runtime lookups, each Trie node caches the Top-5 most frequent completions directly on its node struct. Querying prefix 'sys' reads the precomputed Top-5 array in $O(1)$ time. 3) Offline Aggregation Pipeline: Live search keystrokes stream into Kafka and map-reduce via Apache Flink to aggregate weekly frequency scores. Updated Trie snapshots rebuild offline and hot-swap into distributed memory clusters. 4) Client-Side Optimizations: Frontends implement a 200ms debounce buffer and local LRU cache to slash upstream keystroke queries by 60%.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Top-K Destekli Trie (Önek Ağacı) Implementasyonu
import java.util.*;

public class TypeaheadTrie {
    private static class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        List<String> topSuggestions = new ArrayList<>(); // Önceden hesaplanmış Top-K
    }

    private final TrieNode root = new TrieNode();

    public void insert(String phrase) {
        TrieNode curr = root;
        for (char c : phrase.toLowerCase().toCharArray()) {
            curr = curr.children.computeIfAbsent(c, k -> new TrieNode());
            if (!curr.topSuggestions.contains(phrase)) {
                if (curr.topSuggestions.size() < 3) {
                    curr.topSuggestions.add(phrase);
                }
            }
        }
    }

    public List<String> getSuggestions(String prefix) {
        TrieNode curr = root;
        for (char c : prefix.toLowerCase().toCharArray()) {
            curr = curr.children.get(c);
            if (curr == null) return List.of();
        }
        return curr.topSuggestions;
    }
}

void main() {
    var trie = new TypeaheadTrie();
    trie.insert("sistem tasarimi");
    trie.insert("siber guvenlik");
    trie.insert("sistem gereksinimleri");
    trie.insert("spring boot action");

    IO.println("Öneri ('sis'): " + trie.getSuggestions("sis"));
    IO.println("Öneri ('sp'):  " + trie.getSuggestions("sp"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta Trie ağacını her kullanıcı arama yaptıkça canlı (real-time) olarak güncelleyeceğinizi söylemeyin! Milyonlarca eşzamanlı yazma işlemi Trie ağacında kilitlenme (Lock Contention) yaratır ve belleği tüketir. Güncellemeler daima çevrimdışı batch pipeline ile yapılmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose updating Trie frequencies synchronously on live search queries. Concurrent lock contention on root nodes will crater latency. All frequency score recalculations must occur asynchronously via offline batch pipelines.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Trie (Önek Ağacı) arama süresini kelime sayısından bağımsız olarak O(kelime_uzunluğu) yapar.

• Düğüm üzerinde Top-K önbellekleme arama anında tüm ağacı tarama zorunluluğunu kaldırır.

• İstemci tarafında Debouncing (200ms) uygulamak sunucuya gelen istek yükünü %60 azaltır.

🎯 Key Takeaways for the Interview

• Tries bound search latency to O(prefix_length), completely independent of dictionary scale.

• Pre-computing Top-K suggestions at each Trie node delivers instant O(1) query returns.

• Client-side debouncing buffers rapid typing, cutting keystroke query volume by 60%+.